

UNIVERSIDAD MAYOR DE SAN ANDRES

FACULTAD DE INGENIERIA

INGENIERIA ELECTRONICA

Aplicación de técnicas de control

[ETN-1034]

Planta: Proceso de producción

Integrantes:

Plata Romero Willy Rolando

Gómez Orellana Karel Walter

Sanga Laura Omar Rodrigo

Lovera Callizaya Jhonson

Docente: Ing. Jorge A. Nava A.

La Paz - Bolivia

INVESTIGACIÓN:
Ley de Control y Librería para el Servomotor

Contenido:

1. Introducción....2
2. Funcionamiento del ServoMotor, Driver- PLC....2
   - 2.1. Servomotor....2
   - 2.2. Funcionamiento del husillo....2
   - 2.3. Control mediante el driver y PLC....3
   - 2.4. Ley de Control....5
3. Librería Q0.0 (librería del servomotor)....7
   - 3.1. Bloques de posicionamiento....7
   - 3.2. Q0_x_CTRL....8
   - 3.3. Scale_EU_Pulse....9
   - 3.4. Scale _ Pulse_EU....9
   - 3.5. Q 0_x_Home....10
   - 3.6. Q0_x_MoveRelative....10
   - 3.7. Q0_x_ MoveAbsolute....11
   - 3.8. Q0_x_MoveVelocity....11
   - 3.9. Q0_x_ Stop....12
   - 3.10. Q0_x_ LoadPos....12

1. Introducción

El objetivo de este informe es presentar el funcionamiento de un servomotor que mueve un husillo para trasladar un objeto. Este servomotor es controlado por un driver físico, el cual a su vez es controlado por un PLC (Controlador Lógico Programable).

Es importante destacar las características principales de este sistema y la ley de control que lo gobierna para lograr un rendimiento óptimo.

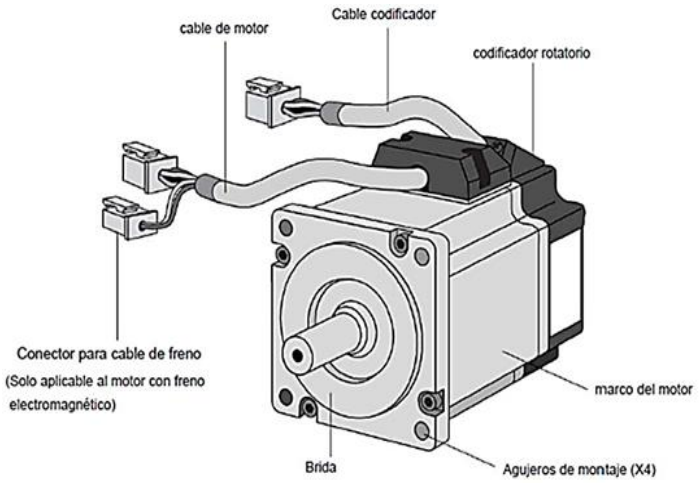
2. Funcionamiento del ServoMotor, Driver- PLC

2.1. Servomotor

Un servomotor es un actuador rotativo o motor que permite un control preciso en términos de posición angular, aceleración y velocidad, capacidades que un motor normal no tiene. En definitiva, utiliza un motor normal y lo combina con un sensor para la retroalimentación de posición. Pero, los servomotores no son en realidad una clase específica de motor, sino una combinación de piezas específicas, que incluyen un motor de corriente continua o alterna, y son adecuados para su uso en un sistema de control de bucle cerrado.

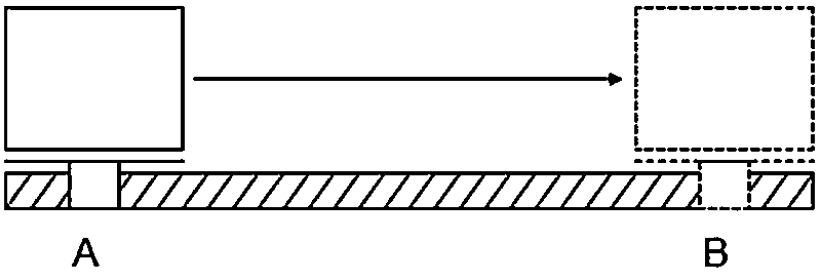
Por lo que una definición más exacta de un servomotor sería la de un servomecanismo de bucle cerrado que utiliza la retroalimentación de posición para controlar su velocidad de rotación y posición. La señal de control es la entrada, ya sea analógica o digital, que representa el comando de posición final para el eje.

Por otro lado, el codificador o encoder sirve como sensor, proporcionando retroalimentación de posición. En la mayoría de los casos, sólo se informa de la posición. La posición final se informa al controlador y se compara con la entrada de posición inicial, y luego, si hay una discrepancia, se mueve el motor para llegar a la posición correcta.



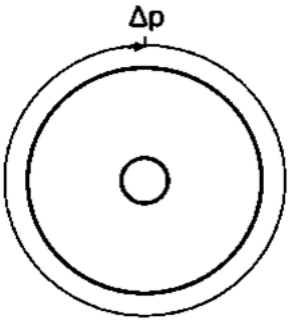
2.2. Funcionamiento del husillo

El husillo es un componente mecánico que se utiliza para convertir el movimiento de rotación del servomotor en movimiento lineal. El servomotor hace girar el husillo, el cual es un tornillo si fin, y este a través de un acoplamiento, este movimiento se transmite al objeto que se desea trasladar.

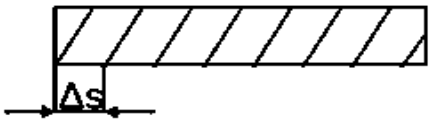


Para nuestro caso en particular, el objetivo es mover un objeto del punto A al punto B, mediante el uso de un husillo (tornillo sin fin) de motor.

Para esto la información que el driver reconoce es solamente la cantidad de pulsos, es decir que nosotros debemos, de alguna manera convertir la señal de pulsos a distancia recorrida por el husillo.



Δp: Número de pulsos en una revolución.



Δs: Distancia recorrida por el husillo en una revolución.

La conversión de posicionamiento de la salida de pulso de la CPU en la distancia recorrida por el husillo (tornillo metálico sin fin) se realiza mediante la referencia a una revolución del motor.

$$p(AB) = \frac{\Delta p}{\Delta s} * AB$$

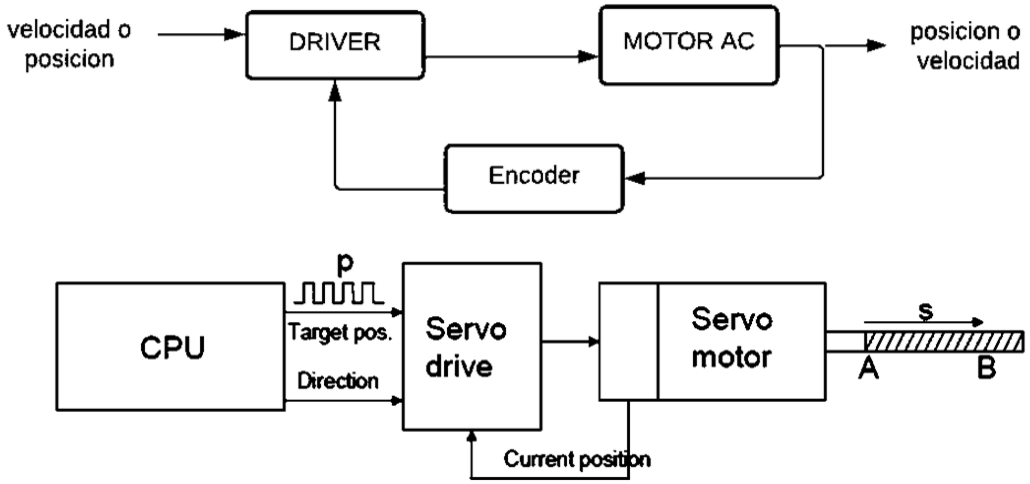
A partir de esta ecuación podemos obtener el dato de la posición que recorrió el objeto desde el punto A hacia el punto B, esta posición es p(AB), que está en unidades de pulsos por revolución.

2.3. Control mediante el driver y PLC

El driver es un dispositivo que actúa como interfaz entre el PLC y el servomotor. Su función principal es recibir las señales enviadas por el PLC y a suministrar la corriente necesaria al servomotor.

El proceso de posicionamiento se implementa utilizando la salida de tren de pulsos del PLC S7-200.

- La CPU envía un tren de pulsos (PTO) y la información direccional (sentido de giro) a un servo-drive.
- El servo-drive convierte la señal en la tensión trifásica en función de la frecuencia (proporcional a la velocidad) y alimenta el servomotor.
- El servomotor devuelve el valor de posición actual para el control de velocidad al servo-drive.



Para una mejor comprensión podemos notar el siguiente ejemplo:

Codificación de posición absoluta:

– El encoder absoluto está compuesto por un disco o una cinta con marcas codificadas, y un conjunto de sensores ópticos o magnéticos que leen esas marcas.

– Cada marca codificada en el disco o cinta representa una posición específica del eje del motor.

– Cuando el servomotor gira, el encoder absoluto detecta y registra la posición absoluta del eje en tiempo real mediante la lectura de las marcas codificadas.

Generación de señales de pulsos:

– El encoder absoluto convierte la información de posición en una secuencia de señales de pulsos.

– Cada pulso representa un incremento o decremento en la posición del eje, y su duración y frecuencia son proporcionales al desplazamiento angular del eje.

– Estas señales de pulsos se envían al driver del servomotor, que las interpreta y las utiliza para controlar el movimiento del motor.

Interpretación de señales por el driver:

– El driver del servomotor está diseñado para reconocer y procesar las señales de pulsos enviadas por el encoder absoluto.

– El driver utiliza la información de posición codificada en los pulsos para determinar la posición actual del eje del motor.

– Basándose en la posición actual y la posición objetivo deseada, el driver calcula la diferencia y genera una señal de corrección para controlar el movimiento del servomotor.

Control del movimiento:

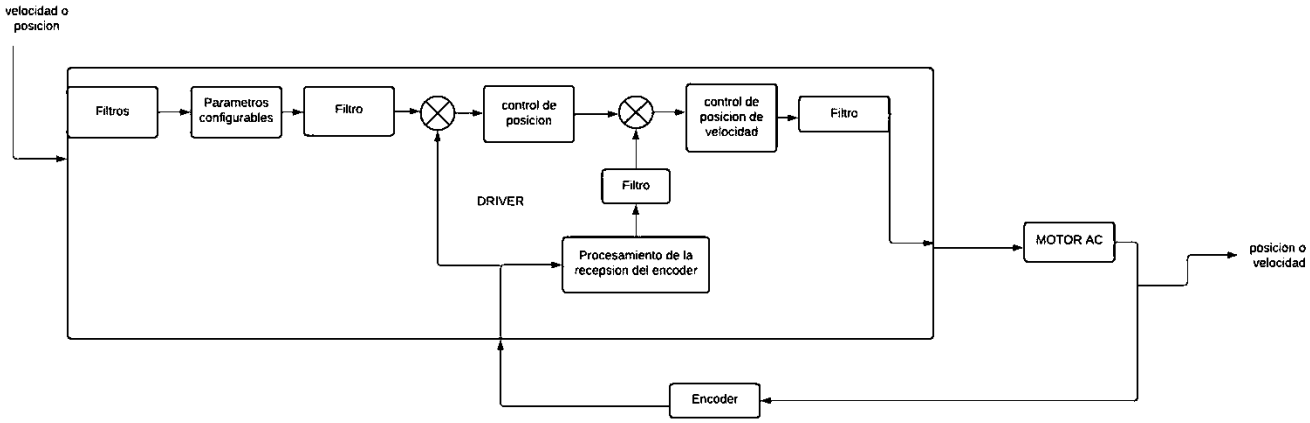
- El driver suministra la corriente adecuada al servomotor para mover el eje en la dirección y velocidad correctas.
- A medida que el servomotor gira, el encoder absoluto continúa generando las señales de pulsos correspondientes a la posición actual del eje.
- El driver monitorea y compara constantemente la posición actual con la posición objetivo, ajustando la corriente suministrada al servomotor para corregir cualquier desviación y mantener la precisión de posicionamiento.

Finalización del movimiento:

- El servomotor se detiene cuando la posición actual alcanza la posición objetivo definida.
- El sistema permanece en reposo hasta recibir una nueva señal de control para iniciar otro movimiento.

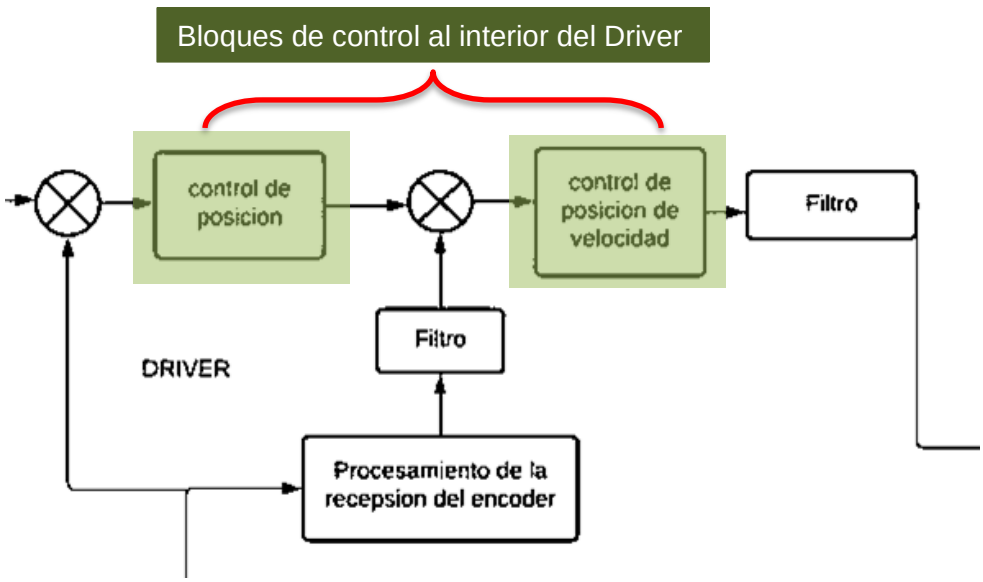
El siguiente diagrama de bloques nos ayudará a comprender mejor el funcionamiento:

DIAGRAMA DE BLOQUES CONCEPTUAL



2.4. Ley de Control

Mediante el anterior diagrama de bloque podemos notar que la ley de control, para gobernar la posición del motor, se encuentra dentro del mismo driver:



Los datos proporcionados en el manual del fabricante nos indican que este encoder absoluto cuenta con 2500 pulsos por cada Revolución. Además, el encoder absoluto es de 7 hilos o bits esto nos permite tener 128 posiciones en una Revolución por cada ¼ de vuelta.

$$D = \frac{\Delta M \times E \times R}{L} = \frac{PR48 \times 2^{PR4A}}{PR4B}$$

$$D = \frac{[resolución\ del\ codificador]}{[número\ de\ pulsos\ por\ revolución]}$$

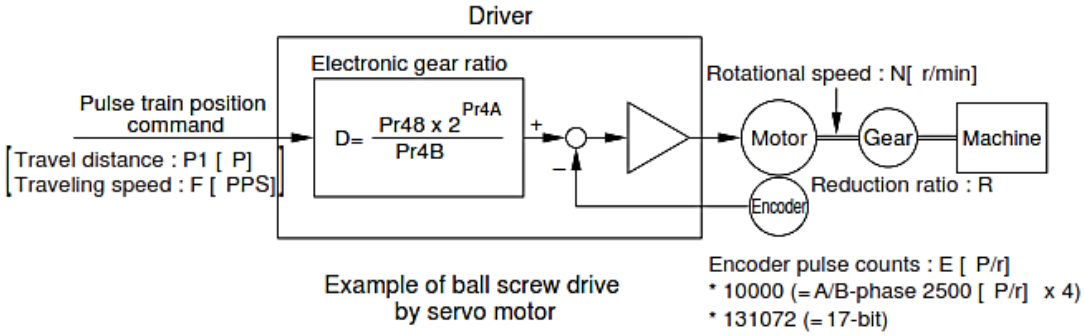
$\Delta M[mm]$: $resolución\ de\ la\ posición$

$E[p/rev]$: $contador\ de\ pulsos$

$R[-]$: $factor\ de\ reducción$

$L[mm]$: $distancia\ del\ husillo$

$$D \geq 1$$
$$D < 1 \rightarrow usar\ 2^{17}$$



Datos:

$Encoder\ = 2500[pulsos/rev]$

$E = Encoder\ \times 4$

$E = 10000\ [pulsos/rev]$

$R = 1$

Ejemplo

Datos:

$L = 10[mm]$

$\Delta M = 0.005\ [mm]$

$$D = \frac{\Delta M \times E \times R}{L} = \frac{0.0005 \times 10000 \times 1}{10} = \frac{10000 \times 1}{200 \times 10} = \frac{10000 \times 2^0}{2000}$$

$$D = \frac{PR48 \times 2^{PR4A}}{PR4B} = \frac{10000 \times 2^0}{2000}$$

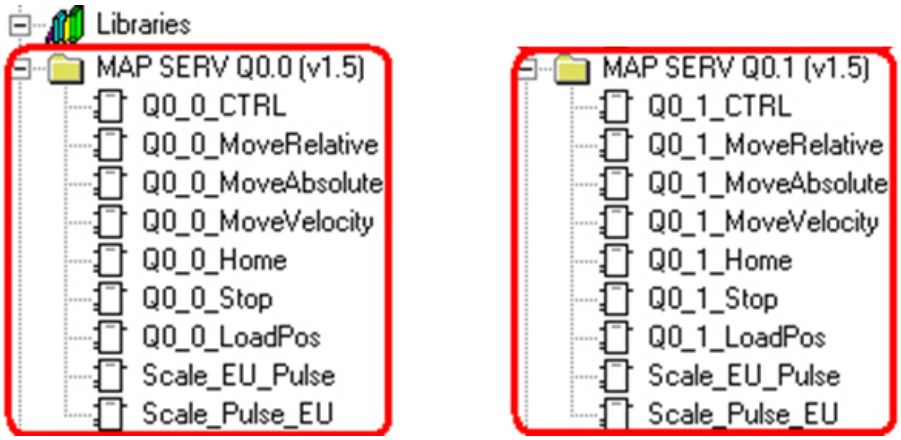
$\boldsymbol{PR48 = 10000}$

$\boldsymbol{PR4A = 0}$

$\boldsymbol{PR4B = 2000}$

3. Librería Q0.0 (librería del servomotor)

La biblioteca “MAP SERV Q0.0”1proporciona los bloques de posicionamiento para la salida Q0.0 y “MAP SERV Q0.1” para Q0.1.



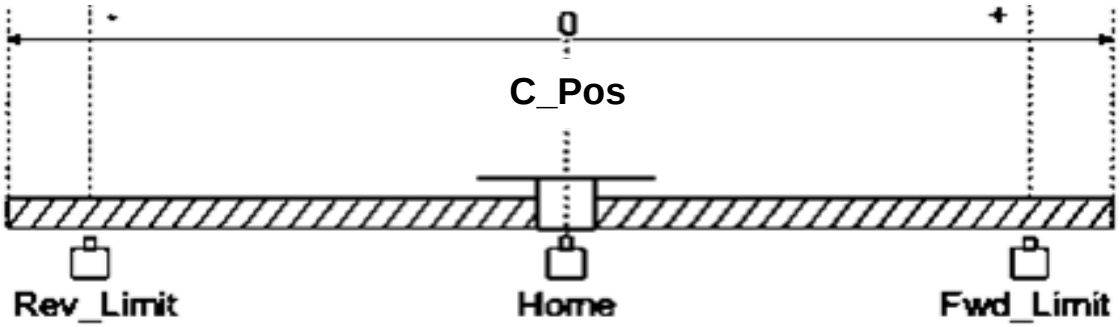
BLOQUE	FUNCIÓN
Q0_x_CTRL	Especificación de parámetros y bloque de control
Q0_x_MoveRelative	Ejecución de un paso de posicionamiento relativo
Q0_x_MoveAbsolute	Ejecución de un paso de posicionamiento absoluto
Q0_x_MoveVelocity	Comenzar un movimiento con velocidad preestablecida
Q0_x_ Home	Buscar la posición del punto de referencia
Q0_x_ Stop	Detener el movimiento
Q0_x_LoadPos	Cargar marcador de posición preestablecido
Scale _UE_Pulse	Conversión de valores de distancia en valores de pulso
Scale_Pulse_EU	Convertir valores de pulso en valores de distancia

3.1. Bloques de posicionamiento

Para utilizar mejor los bloques de posicionamiento, se requieren tres interruptores de límite a lo largo del husillo:

- Un interruptor de punto de referencia (inicio) para el punto cero de la escala de pulsos C_Pos.
- Un interruptor de límite en el sentido positivo (Fwd_Limit) y uno en sentido negativo (Rev_Limit)
- El valor del contador de pulsos C_Pos pertenece al tipo de datos DINT (32 bits)

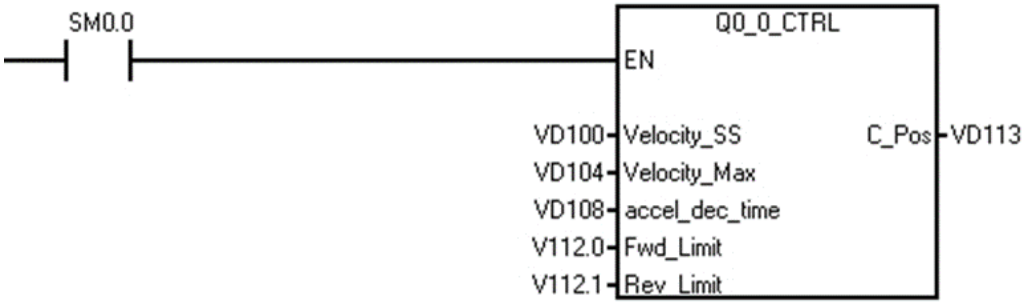
- Los interruptores de final de carrera deben disponerse de modo que todavía haya suficiente espacio la desaceleración.



Esta programación se basa en el tren de impulsos lineal integrado (PTO). Esta función se utiliza para controlar la velocidad. Además, los pulsos se cuentan utilizando el contador de alta velocidad HSC. El punto de inicio de la desaceleración se calcula y activa a través de la interrupción HSC.

3.2. Q0_x_CTRL

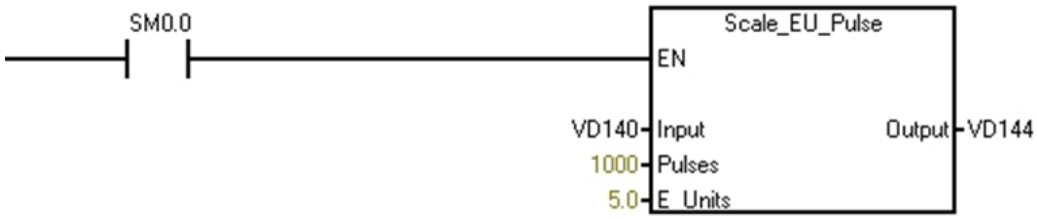
El bloque de control Q0_x_CTRL transfiere los parámetros de posicionamiento global y debe iniciarse cíclicamente.



Parámetro	Tipo	Formato	Unidad	Significado
Velocity_SS	IN	DINT	Pulse/sec.	Frecuencia de inicio/parada
Velocity_Max	IN	DINT	Pulse/sec.	Frecuencia máxima
accel_dec_time	IN	REAL	Sec.	Máxima aceleración y desaceleración tiempo
Fwd_Limit	IN	BOOL		Final de carrera para el recorrido de avance
Rev_Limit	IN	BOOL		Interruptor de límite para el recorrido inverso
C_Pos	OUT	DINT	Pulse.	Posición absoluta actual

3.3. Scale_EU_Pulse

El bloque Scale_EU_Pulse se utiliza para convertir una cantidad de posición en el número correspondiente de pulsos. Entonces, este bloque permite convertir una distancia en el número correspondiente de pulsos o una velocidad en una frecuencia de pulso.



Parámetro	Tipo	Formato	Unidad	Significado
Input	IN	REAL	mm or mm/s.	Distancia o velocidad a convertir
Pulses	IN	DINT	Pulse/ revol	Número de pulsos necesarios para revolución
E_Units	IN	REAL	mm / revol	Distancia recorrida durante una revolución
Output	OUT	DINT	Pulse o pulse/s	Número de pulso equivalente o pulso frecuencia

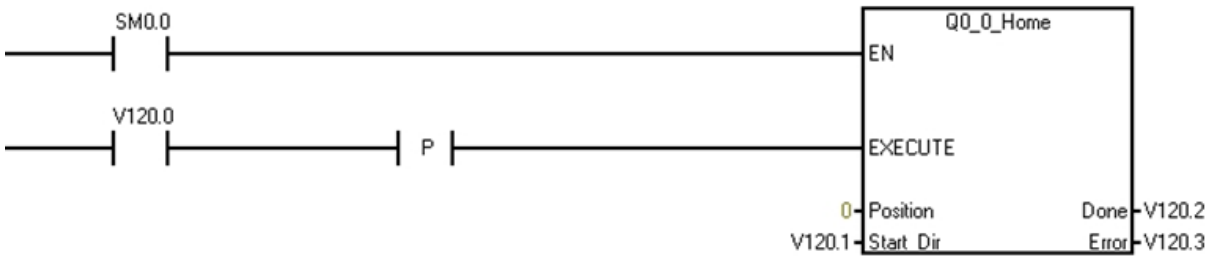
3.4. Scale _ Pulse_EU

El bloque Scale_Pulse_EU se utiliza para convertir una cantidad de pulsos en un valor de distancia.



Parámetro	Tipo	Formato	Unidad	Significado
Input	IN	REAL	Pulse or pulse/s	Número de pulso o frecuencia de pulso a ser convertido
Pulses	IN	DINT	Pulse/ revol	Número de pulsos necesarios para uno revolución
E_Units	IN	REAL	mm / revol	Distancia recorrida durante una revolución
Output	OUT	DINT	mm or mm/s.	Distancia o velocidad correspondiente

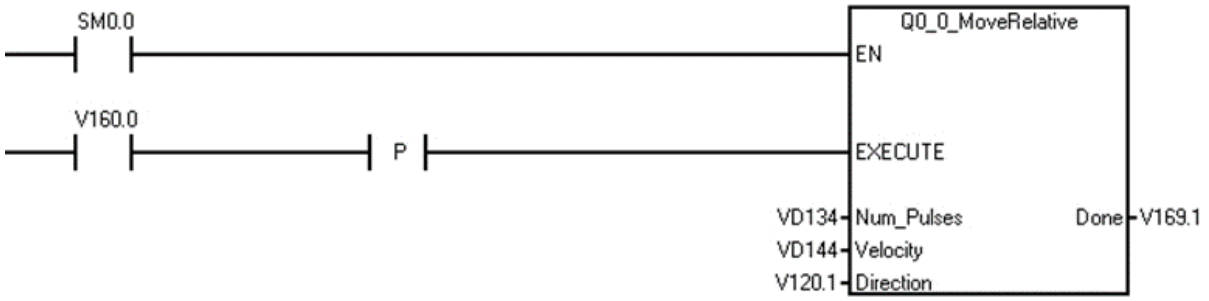
3.5. Q 0_x_Home



Parámetro	Tipo	Formato	Unidad	Significado
EXECUTE	IN	BOOL		Bit de ejecución para el punto de referencia
Position	IN	DINT	Pulse	Valor absoluto de la posición en la referencia punto
Start_Dir	IN	BOOL	Sec.	Dirección el punto de referencia (0=reversa, 1=adelante)
Done	OUT	BOOL		Mensaje listo (1 = hecho)
Error	OUT	BOOL		Mensaje de error (1 = error)

3.6. Q0_x_MoveRelative

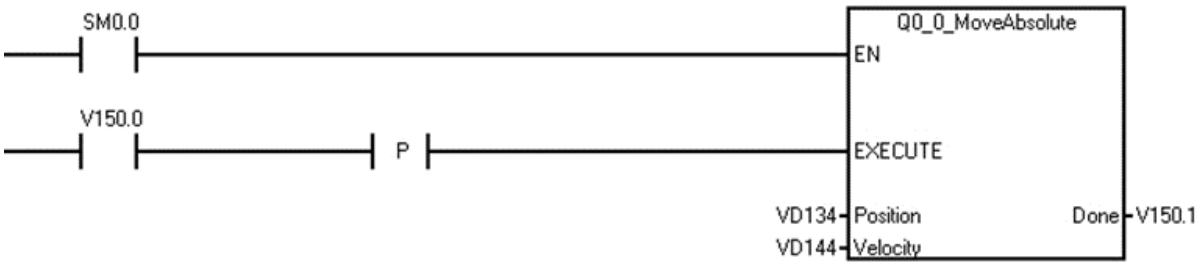
Esta función hace que el eje se mueva por un número de pulso específico en una dirección específica con una frecuencia específica.



Parámetro	Tipo	Formato	Unidad	Significado
EXECUTE	IN	BOOL		Bit de ejecución al posicionamiento relativo proceso
Num_Pulses	IN	DINT	Pulse	Distancia de pulso (debe ser mayor que 1)
Velocity	IN	DINT	Pulse / Sec.	Frecuencia preestablecida (Velocidad_SS <= Velocidad <= Velocidad_máx.)
Direction	IN	BOOL		Dirección preestablecida (0=reversa, 1=adelante)
Done	OUT	BOOL		Mensaje listo (verdadero = hecho)

3.7. Q0_x_ MoveAbsolute

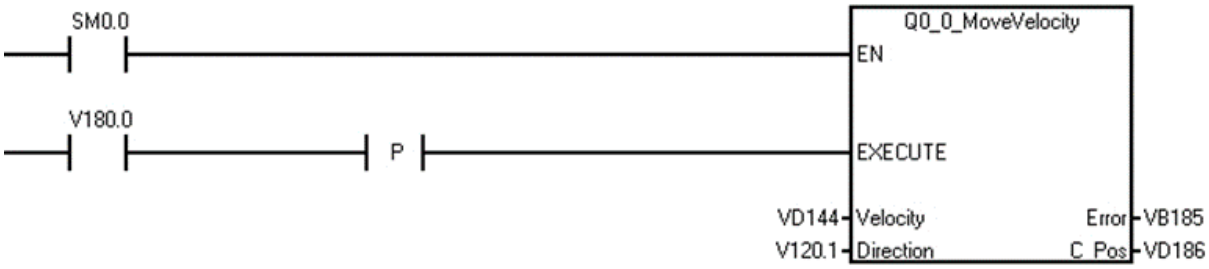
Esta función hace que el eje se mueva a una posición preestablecida (bandera de pulso) con una frecuencia preestablecida.



Parámetro	Tipo	Formato	Unidad	Significado
EXECUTE	IN	BOOL		Bit de ejecución al posicionamiento absoluto proceso
Position	IN	DINT	Pulse	Posicionamiento absoluto
Velocity	IN	DINT	Pulse/Sec.	Frecuencia preestablecida
Done	OUT	BOOL		Mensaje listo (verdadero = hecho)

3.8. Q0_x_MoveVelocity

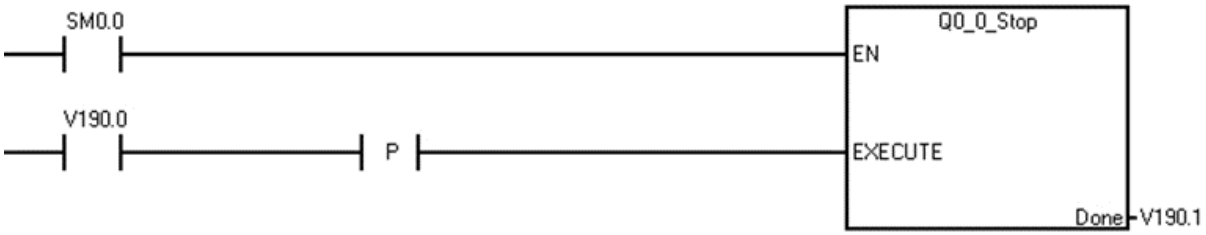
Esta función hace que el eje se mueva en una dirección preestablecida con una frecuencia preestablecida. Es posible un cambio de frecuencia (pero no un cambio de dirección) durante el proceso a través del bit EXECUTE.



Parámetro	Tipo	Formato	Unidad	Significado
EXECUTE	IN	BOOL		bit de ejecución
Velocity	IN	DINT	Pulse/Sec	Frecuencia preestablecida
Direction	IN	BOOL		Dirección preestablecida (0=reversa, 1=adelante)
Error	OUT	BYTE		Mensaje de error (0 = sin error) 1 = parada inmediata sin demora;3 = error de ejecución)
C_Pos	OUT	DINT	Pulse	Posición absoluta actual

3.9. Q0_x_ Stop

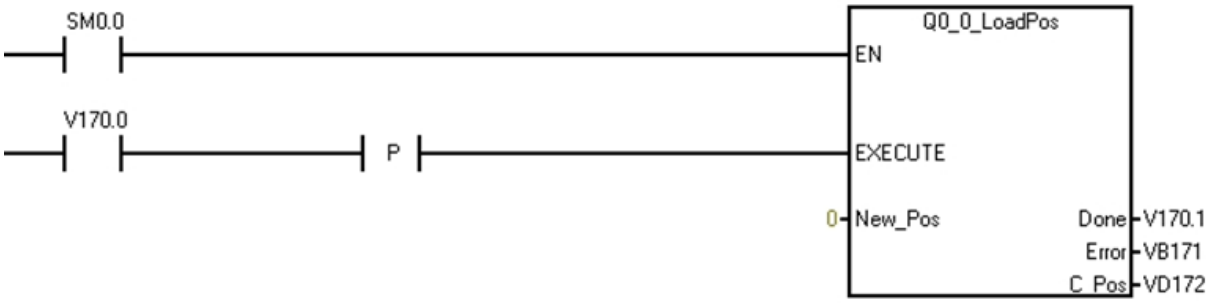
Esta función desacelera el cabezal hasta que se detiene.



Parámetro	Tipo	Formato	Unidad	Significado
EXECUTE	IN	BOOL		bit de ejecución
Done	OUT	BOOL		Mensaje listo (verdadero = hecho)

3.10. Q0_x_ LoadPos

Esta función establece el contador de posición absoluta en un valor predeterminado.



Parámetro	Tipo	Formato	Unidad	Significado
EXECUTE	IN	BOOL		Bit de ejecución para establecer el valor absoluto contador de posicionamiento
New_Pos	IN	DINT	Pulse	Valor de conteo preestablecido
Done	OUT	BOOL	Pulse/Sec	Mensaje listo (verdadero = hecho)
Error	OUT	BYTE		Mensaje de error (0 = sin error)
C_Pos	OUT	DINT	Pulse	Posición absoluta actual